

Al-Mizan Relational Physics Pipelines

SPARC Galaxy Rotation-Curve Calibration & Cosmological Testing Engine
Complete reproducible scripts, numerical results, and data description

This document contains the full Python source for both pipelines, the quantitative outputs obtained on the public SPARC sample (Lelli, McGaugh & Schombert 2016) and the DESI-relevant redshift bins, together with instructions for local reproduction. No model-competitiveness or publication-readiness statements are included.

1. SPARC Calibration Results

Differential-evolution global fit of the three-parameter Mizan acceleration model to 173 SPARC galaxies (2 files discarded for insufficient valid radial points). Objective: mean squared error in $\log_{10}$ g-space (dex^2).

Quantity	Value
Amplitude Normalization $A_{\odot}$	5.234951 ($\log_{10} A_{\odot} = 0.7189$)
Mass-Scaling Power γ	-0.138325
Kernel Exponent K_{global}	0.978294
Global Mean Squared Error	0.0699 dex^2
Mean Residual Bias	-0.0079 dex
Median Absolute Residual	0.1660 dex
Standard Deviation (Scatter)	0.2642 dex
Median Galaxy-by-Galaxy RMS	0.1157 dex
Galaxies used	173

2. Cosmology Pipeline Results (DESI bins)

Parameters: $H_0 = 67.4 \text{ km s}^{-1} \text{ Mpc}^{-1}$, $\Omega_{m0} = 0.315$, $\xi = 0.1$, $L_{\text{decoh}} = 10 \text{ Mpc/h}$, $\Gamma = 4/3$. Growth evaluated at $k = 0.1 \text{ h Mpc}^{-1}$.

BAO distance measures

z	D_M [Mpc]	D_H [Mpc]	D_V [Mpc]
0.15	657.04	4290.54	652.53
0.38	1597.77	3865.28	1553.57
0.51	2082.21	3586.40	1994.16
0.70	2725.09	3184.81	2548.67
0.85	3180.54	2892.48	2918.98
1.48	4688.23	1973.29	4003.96
2.33	6044.30	1292.80	4792.11

Linear growth & RSD ($f\sigma_8$)

z	f(z)	D(z)	$\sigma_8(z)$	$f\sigma_8(z)$
0.15	0.3682	0.9525	0.7725	0.2845
0.38	0.4105	0.8873	0.7196	0.2954
0.51	0.4262	0.8545	0.6930	0.2953
0.70	0.4419	0.8116	0.6582	0.2909
0.85	0.4502	0.7816	0.6339	0.2854
1.48	0.4661	0.6834	0.5542	0.2583
2.33	0.4717	0.5952	0.4827	0.2277

3. SPARC Dataset

Source: SPARC Newtonian mass models (Lelli et al. 2016, AJ 152, 157). File: Rotmod_LTG.zip containing 175 individual *_rotmod.dat files. Columns used: Rad (kpc), Vobs, errV, Vgas, Vdisk, Vbul (km/s). Public download: https://astroweb.case.edu/SPARC/Rotmod_LTG.zip

4. Local Reproduction

```
# 1. Extract SPARC data
unzip Rotmod_LTG.zip -d sparc_data/

# 2. SPARC calibration (requires numpy, pandas, scipy)
python3 mizan_sparc_calibration_clean.py

# 3. Cosmology pipeline
python3 al_mizan_cosmology_pipeline.py
```

5. Source Code — SPARC Calibration

File: `mizan_sparc_calibration_clean.py` (loader fixed for commented SPARC headers; no competitiveness verdict).

```
import os
import glob
import numpy as np
import pandas as pd
from scipy.optimize import differential_evolution

# =====
# 1. ADJUSTED MIZAN RELATIONAL PHYSICS ENGINE
# =====
def compute_log_g_mizan(r, v_baryon, R_c, log10_A0, gamma, K_global, M_bar):
    """
    Computes Mizan modified acceleration profile in log10(m/s^2).
    Uses asymptotic scale-enhancement operator R_k for flat rotation curves.
    """
    conv = 3.24078e-14 # Conversion: (km/s)^2 / kpc -> m/s^2

    # Newtonian baryonic acceleration
    g_baryon = (v_baryon**2) / np.maximum(r, 1e-3) * conv

    # Baryonic mass scaling amplitude A(M_bar)
    M_0 = 1e10
    A_0 = 10**log10_A0
    A_mass = A_0 * np.power(np.maximum(M_bar, 1e6) / M_0, gamma)

    g_mizan = np.zeros_like(r)

    for i, r_val in enumerate(r):
        # Scale step index
        k_step = int(np.ceil(r_val / R_c)) + 1

        # Asymptotic enhancement kernel (prevents outer disk underflow)
        scale_ratio = r_val / (k_step * R_c)
        R_k = 1.0 + A_mass * np.power(scale_ratio, K_global)

        # Mizan accelerated dynamic state
        g_mizan[i] = g_baryon[i] * R_k

    return np.log10(np.maximum(g_mizan, 1e-20))

# =====
# 2. GLOBAL LOG-SPACE OBJECTIVE FUNCTION
# =====
def global_log_loss(params, galaxy_dataset):
    log10_A0, gamma, K_global = params
    total_squared_dex_error = 0.0
    total_points = 0

    for gal in galaxy_dataset:
        r = gal['r']
        log_g_obs = gal['log_g_obs']
        v_disk = gal['v_disk']
        v_gas = gal['v_gas']
        v_bulge = gal['v_bulge']
        R_d = gal['R_d']

        # Quadrature baryonic velocity baseline
        v_baryon = np.sqrt(np.maximum(0, 0.5 * (v_disk**2) + (v_bulge**2) + (v_gas**2)))

        # Total baryonic mass proxy (Solar masses)
        M_bar = np.sum(v_baryon**2 * r) * 2.325e5

        log_g_mizan = compute_log_g_mizan(r, v_baryon, R_d, log10_A0, gamma, K_global, M_bar)

        diff = log_g_obs - log_g_mizan
        total_squared_dex_error += np.sum(diff**2)
        total_points += len(r)

    return total_squared_dex_error / max(total_points, 1)

# =====
# 3. SPARC DATA LOADER & OPTIMIZATION PIPELINE
# =====
def run_mizan_sparc_calibration(data_dir="./sparc_data"):
    sparc_files = glob.glob(os.path.join(data_dir, "*.dat"))
    if not sparc_files:
        print(f"Error: No .dat files found in {data_dir}")
        return

    conv = 3.24078e-14
    galaxy_dataset = []

    for filepath in sparc_files:
```

```

try:
    # SPARC rotmod files have fully commented headers; assign names explicitly
    colnames = ['Rad', 'Vobs', 'errV', 'Vgas', 'Vdisk', 'Vbul', 'SBdisk', 'SBbul']
    df = pd.read_csv(
        filepath, sep=r'\s+', comment='#', engine='python',
        names=colnames, header=None
    )
    df = df.apply(pd.to_numeric, errors='coerce').dropna(subset=['Rad', 'Vobs'])

    if len(df) < 3:
        continue

    r = df['Rad'].values
    v_obs = df['Vobs'].values

    mask = (r > 0) & (v_obs > 0)
    r = r[mask]
    v_obs = v_obs[mask]

    if len(r) < 3:
        continue

    g_obs = (v_obs**2) / r * conv

    galaxy_dataset.append({
        'name': os.path.basename(filepath),
        'r': r,
        'log_g_obs': np.log10(g_obs),
        'v_disk': df['Vdisk'].values[mask],
        'v_gas': df['Vgas'].values[mask],
        'v_bulge': df['Vbul'].values[mask] if 'Vbul' in df.columns else np.zeros_like(r),
        'R_d': np.max(r) / 4.0 if np.max(r) > 0 else 2.0
    })
except Exception:
    continue

print(f"Loaded {len(galaxy_dataset)} galaxies for Mizan framework evaluation...")
print("Executing Differential Evolution global stochastic solver...")

# Optimization Bounds:
# log10_A0 in [-2.0, 2.0] -> A_0 in [0.01, 100.0]
# gamma in [-1.5, 1.5]
# K_global in [0.1, 4.0]
bounds = [(-2.0, 2.0), (-1.5, 1.5), (0.1, 4.0)]

result = differential_evolution(
    global_log_loss,
    bounds,
    args=(galaxy_dataset,),
    strategy='best1bin',
    maxiter=150,
    popsize=20,
    tol=0.005,
    seed=42
)

log10_A0_fit, gamma_fit, K_global_fit = result.x
A0_fit = 10**log10_A0_fit

# Evaluate detailed distribution statistics
delta_all = []
galaxy_scatters = []

for gal in galaxy_dataset:
    v_baryon = np.sqrt(np.maximum(0, 0.5 * (gal['v_disk']**2) + (gal['v_bulge']**2) + (gal['v_gas']**2)))
    M_bar = np.sum(v_baryon**2 * gal['r']) * 2.325e5
    log_g_mizan = compute_log_g_mizan(
        gal['r'], v_baryon, gal['R_d'],
        log10_A0_fit, gamma_fit, K_global_fit, M_bar
    )

    residuals = gal['log_g_obs'] - log_g_mizan
    delta_all.extend(residuals)
    galaxy_scatters.append(np.std(residuals))

delta_all = np.array(delta_all)
galaxy_scatters = np.array(galaxy_scatters)

# Output formatting
print("\n" + "=" * 60)
print("      MIZAN FRAMEWORK: FINAL SPARC EVALUATION RESULTS")
print("=" * 60)
print(f"Amplitude Normalization (A_0) : {A0_fit:.6f} (log10 = {log10_A0_fit:.4f})")
print(f"Mass-Scaling Power (gamma)      : {gamma_fit:.6f}")
print(f"Kernel Exponent (K_global)      : {K_global_fit:.6f}")
print("-" * 60)
print(f"Global Mean Squared Error       : {result.fun:.4f} dex^2")

```

```

print(f"Mean Residual Bias          : {np.mean(delta_all):.4f} dex")
print(f"Median Absolute Residual    : {np.median(np.abs(delta_all)):.4f} dex")
print(f"Standard Deviation (Scatter) : {np.std(delta_all):.4f} dex")
print(f"Median Galaxy-by-Galaxy RMS  : {np.median(galaxy_scatters):.4f} dex")
print("=" * 60)

return {
    'A0': A0_fit, 'log10_A0': log10_A0_fit, 'gamma': gamma_fit,
    'K_global': K_global_fit, 'mse': result.fun,
    'bias': float(np.mean(delta_all)), 'mad': float(np.median(np.abs(delta_all))),
    'scatter': float(np.std(delta_all)), 'median_gal_rms': float(np.median(galaxy_scatters)),
    'n_galaxies': len(galaxy_dataset)
}

if __name__ == "__main__":
    run_mizan_sparc_calibration()

```

6. Source Code — Cosmology Pipeline

File: al_mizan_cosmology_pipeline.py

```
import numpy as np
from scipy.integrate import quad, solve_ivp

class AlMizanCosmologyPipeline:
    def __init__(self, H0=67.4, Om0=0.315, Ob0=0.049, sigma8_0=0.811,
                  xi=1.0, L_decoh=10.0, Gamma=4/3):
        """
        Al-Mizan Al-Mutakamil Cosmological Testing Pipeline Engine.

        Parameters:
        -----
        H0 : float - Present-day Hubble constant (km/s/Mpc)
        Om0 : float - Present-day matter density parameter
        Ob0 : float - Present-day baryon density parameter
        sigma8_0 : float - Amplitude of mass fluctuations at z=0
        xi : float - Relational boundary flux coupling parameter
        L_decoh : float - Environmental decoherence length (Mpc/h)
        Gamma : float - Late-time expansion exponent (Fixed at 4/3)
        """
        self.c = 299792.458 # Speed of light in km/s
        self.H0 = H0
        self.Om0 = Om0
        self.Orel0 = 1.0 - Om0
        self.Ob0 = Ob0
        self.sigma8_0 = sigma8_0
        self.xi = xi
        self.L_decoh = L_decoh
        self.Gamma = Gamma

    def E(self, z):
        """Dimensionless Hubble Parameter E(z) = H(z) / H0"""
        return np.sqrt(self.Om0 * (1 + z)**3 + self.Orel0 * (1 + z)**(-1))

    def H(self, z):
        """Hubble parameter in km/s/Mpc"""
        return self.H0 * self.E(z)

    def dlnH_dlna(self, a):
        """Logarithmic derivative d(ln H) / d(ln a)"""
        z = (1.0 / a) - 1.0
        numerator = 3.0 * self.Om0 * (1 + z)**3 - self.Orel0 * (1 + z)**(-1)
        denominator = self.Om0 * (1 + z)**3 + self.Orel0 * (1 + z)**(-1)
        return -0.5 * (numerator / denominator)

    def G_eff_over_G(self, a, k=0.1):
        """Scale-dependent effective gravitational coupling ratio G_eff(k, z) / G"""
        z = (1.0 / a) - 1.0
        denom = 1.0 + (k**2) * (self.L_decoh**2) * ((1.0 + z)**(-2))
        return 1.0 + (2.0 * (self.xi**2)) / denom

    def comoving_distance_DM(self, z):
        """Transverse comoving distance D_M(z) in Mpc"""
        integrand = lambda zp: 1.0 / self.E(zp)
        res, _ = quad(integrand, 0, z)
        return (self.c / self.H0) * res

    def hubble_distance_DH(self, z):
        """Line-of-sight Hubble distance D_H(z) in Mpc"""
        return self.c / self.H(z)

    def bao_distance_DV(self, z):
        """Spherically averaged BAO distance D_V(z) in Mpc"""
        DM = self.comoving_distance_DM(z)
        DH = self.hubble_distance_DH(z)
        return (z * DH * (DM**2))**(1/3)

    def compute_growth_and_fsigma8(self, z_eval_list, k=0.1):
        """
        Integrates the linear growth ODE from a_init = 0.001 (z=999) to a=1.0 (z=0)
        and computes f(z) and f\sigma_8(z).
        """
        # ODE system for logarithmic scale: y = [f]
        def growth_ode(lna, y):
            a = np.exp(lna)
            f = y[0]

            # Matter density parameter at scale factor a
            E_a_sq = self.Om0 * (a**(-3)) + self.Orel0 * a
            Om_a = (self.Om0 * (a**(-3))) / E_a_sq

            dlnH = self.dlnH_dlna(a)
            G_ratio = self.G_eff_over_G(a, k)
```

```

        df_dlna = 1.5 * Om_a * G_ratio - f**2 - (2.0 - 0.5 * dlnH) * f
        return [df_dlna]

# Initial condition at a_init: Einstein-de Sitter limit f(a_init) ~ 1.0
a_init = 1e-3
lna_init = np.log(a_init)
lna_eval = np.log(1.0 / (1.0 + np.array(z_eval_list)))

# Sort evaluation points monotonically for numerical stability
sort_idx = np.argsort(lna_eval)
lna_eval_sorted = lna_eval[sort_idx]

# Extended integration grid including a=1.0 (lna=0) for normalized D(a)
grid = np.unique(np.concatenate([lna_init, 0.0], lna_eval_sorted)))

sol = solve_ivp(growth_ode, [lna_init, 0.0], [1.0], t_eval=grid, method='RK45', rtol=1e-8, atol=1e-8)

# Reconstruct growth factor D(a) = exp( ∫ (f/a) da ) = exp( ∫ f d(lna) )
lna_grid = sol.t
f_grid = sol.y[0]

# Trapezoidal integration for D(a) relative to z=0
ln_D = np.zeros_like(lna_grid)
for i in range(1, len(lna_grid)):
    ln_D[i] = ln_D[i-1] + 0.5 * (f_grid[i] + f_grid[i-1]) * (lna_grid[i] - lna_grid[i-1])

D_grid = np.exp(ln_D)
D_z0 = D_grid[np.argmin(np.abs(lna_grid - 0.0))]
D_normalized = D_grid / D_z0 # D(z=0) = 1.0

# Map evaluated results back to target redshift array
results = {}
for z_target in z_eval_list:
    lna_target = np.log(1.0 / (1.0 + z_target))
    idx = np.argmin(np.abs(lna_grid - lna_target))

    f_z = f_grid[idx]
    D_z = D_normalized[idx]
    sigma8_z = self.sigma8_0 * D_z
    fsigma8_z = f_z * sigma8_z

    results[z_target] = {
        'f(z)': f_z,
        'D(z)': D_z,
        'sigma8(z)': sigma8_z,
        'fsigma8(z)': fsigma8_z
    }

return results

# =====
# Execution & Verification Run
# =====
if __name__ == "__main__":
    pipeline = AlMizanCosmologyPipeline(H0=67.4, Om0=0.315, xi=0.1, L_decoh=10.0)

    # Target DESI Redshift Bins
    desi_redshifts = [0.15, 0.38, 0.51, 0.70, 0.85, 1.48, 2.33]

    print("=====")
    print(" AL-MIZAN OBSERVATIONAL TESTING PIPELINE: DESI BAO & RSD OUTPUT")
    print("=====")
    print(f"{'z':<6} | {'D_M(z) [Mpc]':<14} | {'D_H(z) [Mpc]':<14} | {'D_V(z) [Mpc]':<14}")
    print("-" * 65)
    for z in desi_redshifts:
        dm = pipeline.comoving_distance_DM(z)
        dh = pipeline.hubble_distance_DH(z)
        dv = pipeline.bao_distance_DV(z)
        print(f"{'z':<6.2f} | {'dm':<14.2f} | {'dh':<14.2f} | {'dv':<14.2f}")

    print("\n=====")
    print(" LINEAR GROWTH & RSD METRICS (f_8) AT k = 0.1 h/Mpc")
    print("=====")
    growth_results = pipeline.compute_growth_and_fsigma8(desi_redshifts, k=0.1)
    print(f"{'z':<6} | {'f(z)':<10} | {'D(z)':<10} | {'sigma8(z)':<10} | {'fsigma8(z)':<10}")
    print("-" * 60)
    for z in desi_redshifts:
        res = growth_results[z]
        print(f"{'z':<6.2f} | {'res['f(z)']':<10.4f} | {'res['D(z)']':<10.4f} | {'res['sigma8(z)']':<10.4f} | {'res['fsigma8(z)']':<10.4f}")

```